HITBOT

hitbot@hitbot.cc
www.hitbot.cc

Z-Arm- API-WINDOWS-PYTHON

领先的轻量型协作机械臂提供商

主营: 工业机器人 /协作机器人 /电动夹爪 /电缸模组

慧灵科技（深圳）有限公司
Huiling-tech Robotic Co., LTD

发布说明

日期	版本	发布说明
2018.07.03	1.0.0.20180703_release	首次发布
2018.08.04	1.0.0.20180814_release	更正 is_stop 返回值定义
20200228	2.0.0.20200228_release	常规更新
20200810	2.0.0.20200810_release	1，添加编码器位置获取接口 2，更正 net_port_initial 返回值的定义 3，添加函数过时提示 4，添加部分新功能函数
20220311	2.0.0.20220311_release	1.增双通道夹爪控制接口； 2.增加硬急停清除接口； 3.增加硬急停状态获取接口；
20221122	2.0.0.20221122_release	1.查询机械臂是否到达目标点；
20231116	2.0.0.20231116_release	1.增加函数调用示例；
20231207	2.0.0.20231207_release	1.增加 485 夹爪控制指令；

目录

1、准备工作 6

2、初始化流程 6

3、SDK 接口 7

3.1 HitbotInterface 类： 7

3.1.1 HitbotInterface 成员变量： 7

3.1.1.1 机械臂当前状态： 7
3.1.1.2 电爪状态 8
3.1.1.3 编码器位置 9

3.1.2 HitbotInterface 成员函数： 9

3.1.2.1 net_port_initial 9
3.1.2.2 close_server 10
3.1.2.3 initial 10
3.1.2.4 get_scara_param 11
3.1.2.5 ~~set_arm_length~~(过时，不推荐使用) 12
3.1.2.6 unlock_position 12
3.1.2.7 is_connect 13
3.1.2.8 get_joint_state 13
3.1.2.9 set_drag_teach 15
3.1.2.10 get_drag_teach 15
3.1.2.11 set_cooperation_fun_state 16
3.1.2.12 get_cooperation_fun_state 16
3.1.2.13 is_collision 17
3.1.2.14 ~~stop_move~~(过时，不推荐使用) 17
3.1.2.15 joint_home 18
3.1.2.16 movel_xyz 18
3.1.2.17 ~~movej_xyz~~(过时，推荐使用 new_movej_xyz_lr) 20
3.1.2.18 ~~movej_angle~~(过时，推荐使用 new_movej_xyz_lr) 21
3.1.2.19 change_attitude 23
3.1.2.20 ~~movel_xyz_by_offset~~(过时，不推荐使用) 24
3.1.2.21 wait_stop 25
3.1.2.22 pause_move 26

3.1.2.23 resume_move 26
3.1.2.24 is_stop 27
3.1.2.25 set_digital_out 27
3.1.2.26 get_digital_out 28
3.1.2.27 get_digital_in 28
3.1.2.28 set_efg_state 29
3.1.2.29 get_efg_state 30
3.1.2.30 ~~movej_xyz_lr~~(过时，推荐使用 new_movej_xyz_lr) 30
3.1.2.31 new_movej_xyz_lr 32
3.1.2.32 new_set_acc 34
3.1.2.33 j5_motor_zero 34
3.1.2.34 set_j5_motor_pos 35
3.1.2.35 get_j5_parameter 36
3.1.2.36 movej_j5 36
3.1.2.37 get_efg_state_dji 37
3.1.2.38 set_efg_state_dji 38
3.1.2.39 new_stop_move 39
3.1.2.40 get_encoder_coor 40
3.1.2.41 new_movej_angle 40
3.1.2.42 new_set_efg_state(新增双通道) 42
3.1.2.43 new_get_efg_state(新增双通道) 43
3.1.2.44 get_hard_emergency_stop_state 44
3.1.2.45 clear_hard_emergency_stop 44
3.1.2.46 is_robot_goto_target 45
3.1.2.47 com485_initial 46
3.1.2.48 com485_send 47
3.1.2.49 com485_recv 48
3.1.2.50 emergency_stop 48
3.1.2.51 judge_in_range_xyzr 48
3.1.2.52 check_joint 49
3.1.2.53 single_joint_move 50
3.1.2.54 xyz_move 51
3.1.2.55 set_Interpolation_position_buf_catch_count 52
3.1.2.56 set_robot_buf_size 52
3.1.2.57 set_robot_joint_torque_value 53
3.1.2.58 set_stop_io 54
3.1.2.59 com485_init 54
3.1.2.60 com485_set_rotation_speed 55
3.1.2.61 com485_set_clamping_speed 55
3.1.2.62 com485_set_clamping_distance 56
3.1.2.63 com485_get_clamping_distance 57

3.1.2.64 com485_set_rotation_angle 57
3.1.2.65 com485_set_relative_rotation_angle 58
3.1.2.66 com485_get_rotation_angle 58
3.1.2.67 com485_get_clamping_state 59
3.1.2.68 com485_get_rotation_state 59
3.1.2.69 com485_set_clamping_current 60
3.1.2.70 com485_set_rotation_current 61
3.1.2.71 com485_get_clamping_current 61
3.1.2.72 com485_get_rotation_current 62
3.1.2.73 com485_suction_cup_control 62
3.1.2.74 com485_min_vacuum_degree 63
3.1.2.75 com485_max_vacuum_degree 64
3.1.2.76 com485_get_draw_state 64
3.1.2.77 get_error_code 65

开发环境：win7 win10 python2/3 x86/x64

1、准备工作

- 复制 HitbotInterface.py 和 small_scara_interface.dll 到源码目录；
- 工程中添加 from HitbotInterface import HitbotInterface；
- 部分开发环境下 net_port_initial 中 CDLL 库目录需要修改成绝对路径；

2、初始化流程

```
from HitbotInterface import HitbotInterface


robot_id = 24
robot = HitbotInterface(robot_id)
robot.net_port_initial()

ret = robot.is_connect()
while ret != 1:
    time.sleep(0.1)
    ret = robot.is_connect()
    print(ret)

ret = robot.initial(1, 210)
if ret == 1:
    print("robot initial successful")
else:
    print("robot initial failed")
```

3、SDK 接口

3.1 HitbotInterface 类：

类名	HitbotInterface
功能	申请内存和初始化网络服务器，控制机械臂。

3.1.1 HitbotInterface 成员变量：

3.1.1.1 机械臂当前状态：

调用 get_scara_param 更新	
变量定义	功能
card_number	机械臂 id 号
X	x 坐标（mm）
Y	y 坐标（mm）
Z	z 坐标（mm）
r	轴 4 角度（deg）
angle1	轴 1 角度（deg）
angle2	轴 2 角度（deg）
communicate_success	• True：不在线 • False：在线
initial_finish	• True：已初始化 • False：未初始化

move_flag	• True：正在运动 • False：静止状态

代码示例：

```
robot.get_scara_param()
print("x=" + robot.x + " y=" + robot.y+ " z=" + robot.z+ " r=" + robot.r+ " angle1=" + robot.angle1+ " angle2=" + robot.angle2 + " communicate_success=" + robot.communicate_success+ " initial_finish=" + robot.initial_finish + " move_flag=" + robot.move_flag)
```

3.1.1.2 电爪状态

调用 get_efg_state 更新	
变量定义	功能
efg_type	• 8 ：当前控制的电爪类型 EFG-8 • 20：当前控制的电爪类型 EFG-20
efg_distance;	efg_type==8： • 255：夹指正在运动 • 0 ：夹爪处于张开状态 • 1 ：夹爪处于闭合状态 efg_type==20：夹指的当前位置

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.get_efg_state()
if ret == 1:
    print("获取成功，夹爪类型="+ robot.efg_type)
    print("获取成功，夹爪当前行程="+ robot.efg_distance)
else:
    print("获取失败")
```

3.1.1.3 编码器位置

调用 get_encoder_coor 更新	
变量定义	功能
encoder_x	编码器 x 轴坐标
encoder_y	编码器 y 轴坐标
encoder_z	编码器 z 轴坐标
encoder_r	编码器 r 轴坐标
encoder_angle1	编码器 angle1 轴坐标
encoder_angle2	编码器 angle2 轴坐标

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
robot.get_encoder_coor()
print("real_x=" + robot.encoder_x + " real_y=" + robot.encoder_y+ " real_z=" + robot.encoder_z+ " real_r=" + robot.encoder_r+ " real_angle1=" + robot.encoder_angle1+ " real_angle2=" + robot.encoder_angle2)
```

3.1.2 HitbotInterface 成员函数：

3.1.2.1 net_port_initial

功能	申请内存和初始化网络服务器
注意	调用其他 api 之前，需要先调用该函数，否则将导致程序崩溃，同一个 python 程序控制多台手臂时，每一个手臂对象都要调用该函数
函数定义	def net_port_initial(self)

参数	无
返回	•1 成功 •0 失败，一般是 40000 端口号被占用

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.net_port_initial()
if   ret == 1:
      print("网络初始化成功")
else:
      print("网络初始化失败")
```

3.1.2.2 close_server

功能	关闭网络服务器(server.exe 程序)
注意	调用后，手臂和 pc 的通讯将会断开
函数定义	def close_server(self)
参数	无
返回	无

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
robot.close_server()
```

3.1.2.3 initial

功能	初始化机械臂的各项参数，机械臂进入工作状态
注意	• 机械臂在运动时调用该函数，会造成机械臂抖动 • 返回 3 时，可以使用 joint_home 使关节强制回零
函数定义	def initial(self, generation, z_trail)
参数	• int generation：400mm 臂展系列传入 1，320mm 臂展系列

	及其他臂展手臂传入 5， • float z_travel：上下关节有效行程（mm）
返回	• 0：机械臂不在线 • 1：初始化成功 • 2：generation 参数错误 • 3：机械臂当前位置不在限定范围，可以使用 joint_home 使关节强制回零 • 12：z_travel 传参错误 • 101：传入参数 NOT A NUMBER • 105：存在某一个关节失效 • >= 10000，pid 自检异常

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.initial(1, 210)
if  ret == 1:
    print("初始化成功")
else:
    print("初始化失败")
```

3.1.2.4 get_scara_param

功能	更新 x，y，z 等成员变量
注意	机械臂不在线时，坐标参数无意义
函数定义	def get_scara_param(self)
参数	无
返回	无

代码示例：

```
robot.get_scara_param()
print("x=" + robot.x + " y=" + robot.y+ " z=" + robot.z+ " r=" + robot.r+ " angle1=" + robot.angle1+ " angle2=" + robot.angle2 + " communicate_success=" + robot.communicate_success+ " initial_finish=" + robot.initial_finish + " move_flag=" + robot.move_flag)
```

3.1.2.5 ~~set_arm_length~~(过时，不推荐使用)

功能	设置机械臂轴 1 和轴 2 长度
注意	initial 函数中会按照设定机型自动设置该参数，如果臂长没有变化，无需重复调用
函数定义	def set_arm_length(self, l1, l2)
参数	• float l1：轴 1 长度（mm） • float l2：轴 2 长度（mm）
返回	无

代码示例：

```
robot.set_arm_length(220,200)
```

3.1.2.6 unlock_position

功能	解锁机械臂，使机械臂可以接受运动指令
注意	一般初始化成功后立刻调用
函数定义	def unlock_position(self)
参数	无
返回	• 0：机械臂不在线 • 1：成功

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.unlock_position()
if   ret == 1:
      print("解锁成功")
else:
      print("机械臂不在线")
```

3.1.2.7 is_connect

功能	查询机械臂是否在线
注意	无
函数定义	def is_connect(self)
参数	无
返回	• False：机械臂不在线 • True：机械臂在线

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.is_connect()
if   ret == True:
      print("机械臂在线")
else:
      print("机械臂不在线")
```

3.1.2.8 get_joint_state

功能	获取机械臂关节状态
注意	无
函数定义	def get_joint_state(self, joint_num)
参数	int joint_num：轴号 1-4
返回	• 0：轴发生复位，需要重新初始化 • 1：关节正常

	• 2：传入参数超范围 • 3：未初始化 • 4：轴状态获取失败 • 5：发生碰撞; • 6：处于拖动模式 • 7：关节缺轴/失效 • 8：编码器错误 • 9：电机堵转/过电流保护 • 10：过压保护 • 11：手臂收到 PC 发送的错误数据 • 12：手臂主控与驱动通讯异常 • 21： MOS ntc 开路 • 22：MOS ntc 短路 • 23：电机 ntc 开路 • 24：电机 ntc 短路 • 25：MOS ntc 温度过高 • 26：MOS ntc 温度过低 • 27：电机 ntc 温度过高 • 28：电机 ntc 温度过低

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.get_joint_state(1)
if   ret == 1:
      print("关节正常")
else:
```

```
    print("关节异常")
```

3.1.2.9 set_drag_teach

功能	设定拖动示教功能是否开启
注意	• 仅协作机型支持开启此功能 • 拖动示教功能开启后，轴 3 不支持拖动，需要通过调用运动函数控制轴 3， • 部分新机型支持 z 轴拖动示教
函数定义	def set_drag_teach(self, enable)
参数	bool enable：True 开启，False 关闭
返回	• False：失败 • True：成功

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.set_drag_teach_state(True)
if  ret == True:
    print("拖动示教开启成功")
else:
    print("拖动示教开启失败")
```

3.1.2.10 get_drag_teach

功能	查询是否处于拖动示教状态
注意	无
函数定义	def get_drag_teach(self)
参数	无
返回	• False：开启

	• True：关闭

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.get_drag_teach_state()
if   ret == True:
      print("拖动示教关闭")
else:
      print("拖动示教开启")
```

3.1.2.11 set_cooperation_fun_state

功能	设定协作功能是否开启，开启后机械臂遇到障碍物会停止运动，并上报此状态
注意	仅协作机型支持开启此功能
函数定义	def set_cooperation_fun_state(self, enable)
参数	bool enable：True 开启，False 关闭
返回	• False：成功 • True：失败

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.set_cooperation_fun_state(True)
if   ret == True:
      print("设定协作功能开启失败")
else:
      print("设定协作功能开启成功")
```

3.1.2.12 get_cooperation_fun_state

功能	查询机械臂是否开启协作功能
注意	
函数定义	def get_cooperation_fun_state(self)

参数	无
返回	• False：开启 • True：关闭

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.get_cooperation_fun_state()
if   ret == True:
      print("设定协作功能关闭")
else:
      print("设定协作功能开启")
```

3.1.2.13 is_collision

功能	查询机械臂是否发生碰撞
注意	无
函数定义	def is_collision(self)
参数	无
返回	• False：未发生碰撞 • True：发生碰撞

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.is_collision()
if   ret == True:
      print("发生碰撞")
else:
      print("未发生碰撞")
```

3.1.2.14 ~~stop_move~~(过时，不推荐使用)

功能	立即结束当前正在执行的指令
注意	无

函数定义	def stop_move(self)
参数	无
返回	无

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
robot.stop_move()
```

3.1.2.15 joint_home

功能	使轴回到零位
注意	无
函数定义	def joint_home(self,joint_num)
参数	• int joint_num：轴号
返回	• 0：未连接 • 1：调用成功 • 2：传入参数错误 • 3：机械臂正在初始化

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.joint_home(1)
if   ret == 1:
      print("回零成功")
else:
      print("回零失败")
```

3.1.2.16 movel_xyz

功能	以 movel 的运动方式到达目标点
注意	• move_flag 等于 False 时，进行首次调用，返回 1 后，可

	以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，再次设定新的目标，机械臂将以一定的速度通过当前目标点，而后去往新的目标点。
函数定义	def movel_xyz(self, goal_x, goal_y, goal_z, goal_r, speed)
参数	• float goal_x：目标点 x 坐标（mm） • float goal_y：目标点 y 坐标（mm） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s）
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：过程点无法到达 • 6：伺服未开启 • 7：存在中间过程点无法以机械臂当前姿态（手系）达到 • 11：手机端在控制 • 99：急停中 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.movel_xyz(250, -80, -150, 0, 100)
if   ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
      print("机械臂指令无法执行")
```

3.1.2.17 ~~movej_xyz~~(过时，推荐使用 new_movej_xyz_lr)

功能	以 movej 的运动方式到达目标点
注意	• move_flag 等于 False 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目标点附近通过，而后去往新的目标点。
函数定义	def movej_xyz(self, goal_x, goal_y, goal_z, goal_r, speed, roughly)
参数	• float goal_x：目标点 x 坐标（mm） • float goal_y：目标点 y 坐标（mm） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将在当前目标点附近以最大速度通过，此参数取值范围 0 到 1，

	越大通过速度越快，但是通过点距离当前目标点越远。
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达 • 6：伺服未开启 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.movej_xyz(250, -80, -150, 0, 100,0)
if   ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
      print("机械臂指令无法执行")
```

3.1.2.18 ~~movej_angle~~(过时，推荐使用 new_movej_xyz_lr)

功能	以 movej 的运动方式到达目标点
注意	• move_flag 等于 False 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目

	标点附近通过，而后去往新的目标点。
函数定义	def movej_angle(self, goal_angle1, goal_angle2, goal_z, goal_r, speed, roughly)
参数	• float goal_angle1：目标点轴 1 角度坐标（deg） • float goal_angle2：目标点轴 2 角度坐标（deg） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将在当前目标点附近以最大速度通过，此参数取值范围 0 到 1，越大通过速度越快，但是通过点距离当前目标点越远。
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达 • 6：伺服未开启 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效

	• 103：轴发生复位，需要重新初始化，本次指令无效

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.movej_angle(50, 50, -100, 0, 100, 0)
if   ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
      print("机械臂指令无法执行")
```

3.1.2.19 change_attitude

功能	切换手系
注意	
函数定义	def change_attitude(self, speed)
参数	float speed：线速度（mm/s）
返回	• 0：机械臂正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：过程点无法到达 • 6：伺服未开启 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.change_attitude(100)
```

```
if  ret == 1:
    print("本次指令生效，机械臂开始运动")
else:
    print("机械臂指令无法执行")
```

3.1.2.20 ~~movel_xyz_by_offset~~(过时，不推荐使用)

功能	以 movel 的运动方式在指定方向上移动指定距离
注意	• move_flag 等于 False 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，再次设定新的目标，机械臂将以一定的速度通过当前目标点，而后去往新的目标点。
函数定义	def movel_xyz_by_offset(self, x_offset, y_offset, z_offset, r_offset, speed)
参数	• float x_offset：目标点 x 坐标（mm） • float yoffset：目标点 y 坐标（mm） • float zoffset：目标点 z 坐标（mm） • float r_offset：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s）
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：过程点无法到达 • 6：伺服未开启

	• 7：存在中间过程点无法以机械臂当前姿态（手系）达到 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.movel_xyz_by_offset(10, -30, -20, 0, 100)
if   ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
      print("机械臂指令无法执行")
```

3.1.2.21 wait_stop

功能	用于判断运动是否结束
注意	• wait_stop 属于开环控制，函数返回以后不代表手臂一定处于静止状态，在关键点位需要搭配 is_robot_goto_target 使用，此时 is_robot_goto_target 需要做超时处理，一般超时时间 3s 即可 • 阻塞函数，运动数据下发完成后返回
函数定义	def wait_stop(self)
参数	无
返回	• true：正常停止 • false：存在某些异常使 wait_stop 超时,此时应停止手臂作业流程

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.wait_stop()
if   ret == True:
      print("正常停止")
else:
      print("存在某些异常使 wait_stop 超时,此时应停止手臂作业流程")
```

3.1.2.22 pause_move

功能	暂停机械臂的运动
注意	函数返回后，经过一定延时（几百毫秒），机械臂停止运动
函数定义	def pause_move(self):
参数	无
返回	• 默认返回 1

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.pause_move()
```

3.1.2.23 resume_move

功能	恢复机械臂的运动
注意	无
函数定义	def resume_move(self)
参数	无
返回	• 默认返回 1

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.resume_move()
```

3.1.2.24 is_stop

功能	非阻塞，查询手臂是否停止运动
注意	无
函数定义	def is_stop(self)
参数	无
返回	• 0：运动 • 1：停止

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.is_stop()
```

3.1.2.25 set_digital_out

功能	设定 io 输出点状态
注意	
函数定义	def set_digital_out(self, io_number, io_value)
参数	• int io_number：io 输出点编号 • bool value：设定值
返回	• 0：io_number 参数错误 • 1：设置成功 • 3：未初始化;

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.set_digital_out(1,true)
if   ret == 1:
      print("设置成功")
else:
```

```
    print("设置失败")
```

3.1.2.26 get_digital_out

功能	获取 io 输出点状态
注意	
函数定义	def get_digital_out(self, io_number)
参数	int io_number：io 输出点编号
返回	• -1：io_number 参数错误 • 0：io 口输出状态为断开 • 1：io 口输出状态为导通 • 3：未初始化

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.get_digital_out(1)
if  ret == 1:
    print("io 口输出状态为导通")
else:
    print("io 口输出状态为断开")
```

3.1.2.27 get_digital_in

功能	获取 io 输入点状态
注意	
函数定义	def get_digital_in(self, io_number)
参数	int io_number：io 输入点编号
返回	• -1：io_number 参数错误 • 0：io 输入点没有被触发

	• 1：io 输入点被触发 • 3：未初始化

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.get_digital_in(1)
if  ret == 1:
    print("io 输入点被触发")
else:
    print("io 输入点没有被触发")
```

3.1.2.28 set_efg_state

功能	设定电动夹爪状态
注意	切换控制类型需要重启机械臂和 sdk
函数定义	def set_efg_state(self, efg_type, efg_distance)
参数	• int type：8 EFG-8，20 EFG-20 • float distance： Type==8： 0 张开 1 闭合 Type==20：实际行程（mm）
返回	• -1：控制参数发生变化 • 0：type 参数错误 • 1：设置成功 • 3：未初始化;

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.set_efg_state(20,0)
```

```
if   ret == 1:
     print("设置成功")
else:
     print("设置失败")
```

3.1.2.29 get_efg_state

功能	获取电动夹爪状态
注意	• 返回 1，更新成员变量中的 efg_type 和 efg_distance • 1632 机型不支持在运动过程中调用
函数定义	def get_efg_state(self)
参数	无
返回	• 1：成功 • 3：失败，未初始化;

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.get_efg_state()
if   ret == 1:
     print("获取成功，夹爪类型="+ robot.efg_type)
     print("获取成功，夹爪当前行程="+ robot.efg_distance)
else:
     print("获取失败")
```

3.1.2.30 ~~movej_xyz_lr~~(过时，推荐使用 new_movej_xyz_lr)

功能	以 movej 的运动方式，并以指定手系达目标点
注意	• move_flag 等于 false 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目

	标点附近通过，而后去往新的目标点。
函数定义	def movej_xyz_lr(self, goal_x, goal_y, goal_z,goal_r,speed, roughly, lr)
参数	• float goal_x：目标点 x 坐标（mm） • float goal_y：目标点 y 坐标（mm） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将在当前目标点附近以最大速度通过，此参数取值范围 0 到 1，越大通过速度越快，但是通过点距离当前目标点越远。 • int lr：1 右手系，-1 左手系，angle2>0 右手系，否则为左手系
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达 • 6：伺服未开启 • 7：无法以设定手系到达目标点位 • 11：手机端在控制

	• 99：急停中 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.movej_xyz_lr(250, -80, -150, 0, 100,0,-1)
if   ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
      print("机械臂指令无法执行")
```

3.1.2.31 new_movej_xyz_lr

功能	以 movej 的运动方式，并以指定手系达目标点
注意	• move_flag 等于 false 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目标点附近通过，而后去往新的目标点。
函数定义	def new_movej_xyz_lr(self,goal_x,goal_y,goal_z,goal_r,speed, roughly,lr)
参数	• float goal_x：目标点 x 坐标（mm） • float goal_y：目标点 y 坐标（mm） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度

	（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将在当前目标点附近以最大速度通过，此参数取值范围 0 到 1，越大通过速度越快，但是通过点距离当前目标点越远。 • int lr：1 右手系，-1 左手系，angle2>0 右手系，否则为左手系
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达 • 6：伺服未开启 • 7：无法以设定手系到达目标点位 • 11：手机端在控制 • 99：急停中 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.new_movej_xyz_lr(250, -80, -150, 0, 100,0,-1)
if   ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
```

```
print("机械臂指令无法执行")
```

3.1.2.32 new_set_acc

功能	设定 new_movej_xyz_lr 运动模型的关节加速度百分比
注意	无，范围>=30,<=220
函数定义	def new_set_acc(self,j1_max_acc,j2_max_acc,j3_max_acc, j4_max_acc)
参数	• int j1_max_acc：关节 1 加速度百分比 • int j2_max_acc：关节 2 加速度百分比 • int j3_max_acc：关节 3 加速度百分比 • int j4_max_acc：关节 4 加速度百分比
返回	• 1：默认返回 1

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.new_set_acc(120, 120, 120, 120)
```

3.1.2.33 j5_motor_zero

功能	1）搭配步进电机扩展模块使用，调用后会将当前位置作为零位 2）搭配第五轴使用时，无效
注意	无
函数定义	def j5_motor_zero(self)
参数	无
返回	• 0：手臂未连接

	• 1：设定成功 • 3：手臂未初始化 • 4：不支持该功能

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.j5_motor_zero()
if   ret == 1:
      print("设定成功")
else:
      print("设定失败")
```

3.1.2.34 set_j5_motor_pos

功能	1）搭配步进电机扩展模块使用，设定步进电机的目前位置，范围限制正负 1000 圈 2）搭配第五轴使用时，范围正负 150 度
注意	手臂本体四个关节在运动时，禁止调用此接口
函数定义	def set_j5_motor_pos(self,deg, speed);
参数	• float deg：目标角度 单位 deg • float speed：平均速度 单位 deg/s
返回	• 0：其他运动函数调用中 • 1：设定成功 • 2：speed 小于等于零 • 3：未初始化 • 4：手臂当前 xyzr 超出运动范围 • 99：急停中

	• 102：手臂发生碰撞 • 103：关节复位 • 105：关节异常 • 106：不支持该功能

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.set_j5_motor_pos(60,100)
if   ret == 1:
      print("设定成功")
else:
      print("设定失败")
```

3.1.2.35 get_j5_parameter

功能	搭配步进电机扩展模块或第五轴使用，返回电机电机当前位置
注意	无
函数定义	def get_j5_parameter(self);
参数	无
返回	• 返回五轴当前位置

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.get_j5_parameter()
```

3.1.2.36 movej_j5

功能	搭配第五轴使用
注意	第五轴旋转范围正负 150deg
函数定义	def movej_j5(self, deg, speed);

参数	• float deg：目标角度 单位 deg • float speed：平均速度 单位 deg/s
返回	• 0：其他运动函数调用中 • 1：设定成功 • 2：speed 小于等于零 • 3：未初始化 • 4：手臂当前 xyzr 超出运动范围 • 99：急停中 • 102：手臂发生碰撞 • 103：关节复位 • 105：关节异常 • 106：不支持该功能

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.movej_j5(60, 100)
if   ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
      print("机械臂指令无法执行")
```

3.1.2.37 get_efg_state_dji

功能	搭配 EFG-NMA 电动夹爪上电夹指不动版本使用，用于查询夹爪位置
注意	仅用于控制上电夹指不动 EFG-NMA 电动夹爪，上电位置为 0

函数定义	def get_efg_state_dji(self)
参数	无
返回	• 返回夹爪当前类型和夹指当前位置

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.get_efg_state_dji()
if   ret == 1:
      print("获取成功，夹爪类型="+ robot.efg_type)
      print("获取成功，夹爪当前行程="+ robot.efg_distance)
else:
      print("获取失败")
```

3.1.2.38 set_efg_state_dji

功能	搭配 EFG-NMA 电动夹爪上电夹指不动版本使用，用于控制夹爪移动
注意	仅用于控制上电夹指不动 EFG-NMA 电动夹爪，上电位置为 0，当电动夹爪是 EFG-20NMA 或 EFG-50NMA 时，需要进行上电校准操作，EFG-20NMA 执行 set_efg_state_dji (20, -1000)上电校准操作，EFG-50NMA 执行 set_efg_state_dji (50, -1000)上电校准操作
函数定义	def set_efg_state_dji(self, type, distance)
参数	• int type：8:EFG-8 20:EFG-20NMA 50:EFG-50NMA • float distance：type==8：0 张开 1 闭合 type==20：实际行程（mm），范围 0~20 type==50：实际行程（mm），范围 0~50
返回	• -10：急停中

	• -1：控制参数发生变化 • 0：type 参数错误 • 1：设置成功 • 2: 1235 端口长时间被占用超时或者 1235 端口查询线程挂掉超时 • 3：未初始化 • 4：50nma 夹爪控制机型错误 • 5：爪子运行出错 • 101：传入参数 NOT A NUMBER

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.set_efg_state_dji(20,0)
if   ret == 1:
      print("设置成功")
else:
      print("设置失败")
```

3.1.2.39 new_stop_move

功能	立即结束当前正在执行的指令
注意	相比 stop_move，优化了停止算法，建议搭配 new_movej_xyz_lr 使用
函数定义	def new_stop_move(self)
参数	无
返回	无

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
robot.new_stop_move()
```

3.1.2.40 get_encoder_coor

功能	更新编码器位置，调用后 encoder_x,encoder_y,encoder_z 等变量会进行更新
注意	
函数定义	def get_encoder_coor(self)
参数	无
返回	无

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
robot.get_encoder_coor()
print("real_x=" + robot.encoder_x + " real_y=" + robot.encoder_y+ " real_z=" + robot.encoder_z+ " real_r=" + robot.encoder_r+ " real_angle1=" + robot.encoder_angle1+ " real_angle2=" + robot.encoder_angle2)
```

3.1.2.41 new_movej_angle

功能	以 movej 的运动方式到达目标点
注意	• move_flag 等于 False 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目标点附近通过，而后去往新的目标点。
函数定义	def new_movej_angle(self, goal_angle1, goal_angle2, goal_z, goal_r, speed, roughly)
参数	• float goal_angle1：目标点轴 1 角度坐标（deg）

	• float goal_angle2：目标点轴 2 角度坐标（deg） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将在当前目标点附近以最大速度通过，此参数取值范围 0 或 1
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达 • 6：伺服未开启 • 11：手机端在控制 • 99：急停中 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.new_movej_angle(50, 50, -100, 0, 100, 0)
if   ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
```

print("机械臂指令无法执行")

3.1.2.42 new_set_efg_state(新增双通道)

功能	设定电动夹爪状态
注意	•切换控制类型需要重启机械臂和 sdk,当电动夹爪是 EFG-20NMA、EFG-30NMA 或 EFG-50NMA 时，需要进行上电校准操作，EFG-20NMA 执行 new_set_efg_state (channel,-20, -1000) 上电校准操作，EFG-30NMA 执行 new_set_efg_state (channel,-30, -1000)上电校准操作，EFG-50NMA 执行 new_set_efg_state (channel,-50, -1000)上电校准操作
函数定义	def new_set_efg_state(self, channel, type, distance)
参数	• int channel：0: 通道 0，1: 通道 1， • int type：8: EFG-8， 20: EFG-20NM -20:EFG-20NMA 30:EFG-30NM -30:EFG-30NMA 50:EFG-50NM -50:50NMA • float distance： Type==8： 0 张开 1 闭合 Type==20，-20，30，-30，50，-50：实际行程（mm）
返回	• -10：急停中

	• -1：该机型不支持 • 0：type 参数错误 • 1：设置指令成功 • 2：输入参数错误 • 3：未初始化 • 101：参数异常，被篡改

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.new_set_efg_state(0,20,0)
if   ret == 1:
      print("设置成功")
else:
      print("设置失败")
```

3.1.2.43 new_get_efg_state(新增双通道)

功能	获取电动夹爪状态
注意	位置值存储在全局变量 efg_distance 中
函数定义	def new_get_efg_state (self,channel,type)
参数	• int channel : 0: 通道 0，1: 通道 1， • int type：电爪类型
返回	• -1：该机型不支持； • 1：获取成功； • 2：输入参数错误； • 3：未初始化；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
```

```
ret = robot.new_get_efg_state(0,20)
if   ret == 1:
      print("获取成功，夹爪当前行程="+ robot.efg_distance)
else:
      print("获取失败")
```

3.1.2.44 get_hard_emergency_stop_state

功能	获取急停状态
注意	目前 4150,4160 新机型支持
函数定义	def get_hard_emergency_stop_state(self)
参数	无
返回	-1：该机型不支持； 0：未处于急停状态； 1：处于急停状态； 3：未连接； 4：没有急停功能；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.get_hard_emergency_stop_state()
if   ret == 1:
      print("处于急停状态")
else:
      print("未处于急停状态")
```

3.1.2.45 clear_hard_emergency_stop

功能	急停按钮复位后，发送清除急停状态，进入正常模式
注意	目前 4150,4160 新机型支持
函数定义	def clear_hard_emergency_stop(self)

参数	无
返回	-1：该机型不支持； 0：未处于急停状态； 1：解除成功； 1：解除超时 3：未连接； 4：没有急停功能； 5：正处于急停状态

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.clear_hard_emergency_stop()
if   ret == 1:
      print("解除成功")
else:
      print("解除失败")
```

3.1.2.46 is_robot_goto_target

功能	查询机械臂是否到达目标点
注意	无
函数定义	def is_robot_goto_target(self, target_x, target_y, target_z, target_r)
参数	• float target_x：目标点 x 坐标（mm） • float target_y：目标点 y 坐标（mm） • float target_z：目标点 z 坐标（mm） • float target_r：目标点 r 坐标（deg）
返回	0：未到达；

	1：已到达；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.is_robot_goto_target(250, -80, -150, 0)
if   ret == 1:
      print("已到达")
else:
      print("未到达")
```

3.1.2.47 com485_initial

功能	RS485 夹爪初始化设置波特率
注意	无
函数定义	def com485_initial(self, baudRate)
参数	• int baudRate：波特率
返回	1：夹爪初始化成功； 2：未初始化； 3：未连接； 4：波特率过小； 5：波特率过大； 6：接收超时； 7：设定反馈超时； 8：非 485 扩展模块； -1：I/O 板发送超时；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.com485_initial(115200)
if   ret == 1:
```

```
    print("夹爪初始化成功")
else:
    print("夹爪初始化失败")
```

3.1.2.48 com485_send

功能	RS485 夹爪发送数据控制夹爪
注意	无
函数定义	def com485_send(self, data,len)
参数	• unsigned char *data：485 夹爪数据 • unsigned char len：485 夹爪数据长度
返回	1：夹爪发送数据成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
data=b'\x01\x10\x00\x02\x00\x02\x04\x00\x00\x00\x00\x72\x76'
ret = robot.com485_send(data,13)
if  ret == 1:
    print("夹爪发送数据成功")
else:
    print("夹爪发送数据失败")
```

3.1.2.49 com485_recv

功能	RS485 夹爪接收返回数据
注意	无
函数定义	def com485_recv(self, data)
参数	• unsigned char *data：485 夹爪返回数据
返回	8：夹爪返回数据长度； -1：非 485 扩展模块； 0：485 接收失败；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
data=b'\x01\x10\x00\x02\x00\x02\x04\x00'
return robot.com485_recv(data)
print("夹爪接收数据"+data)
```

3.1.2.50 emergency_stop

功能	立刻停止机械臂运动，然后释放除上下以外关节的转矩
注意	无
函数定义	def emergency_stop(self)
参数	• 无
返回	无

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
robot.emergency_stop()
```

3.1.2.51 judge_in_range_xyzr

功能	判断点位是否可以到达

注意	无
函数定义	def judge_in_range_xyzr(self,x,y,z,r)
参数	• float x：x 坐标(mm) • float y：y 坐标(mm) • float z：z 坐标(mm) • float r：r 坐标(deg)
返回	false：超出范围无法到达； true：可以到达；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.judge_in_range_xyzr(250, -80, -150, 0)
if   ret == True:
      print("可以到达")
else:
      print("超出范围无法到达")
```

3.1.2.52 check_joint

功能	检查关节是否有效
注意	无
函数定义	def check_joint(self,joint_num,state)
参数	• int joint_num：关节号 1-4 • bool state：true 开启关节 false 屏蔽
返回	true：设置关节是否生效成功； false：关节号错误；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
```

```
ret = robot.check_joint(4, False)
if   ret == True:
      print("设置关节是否生效成功")
else:
      print("关节号错误")
```

3.1.2.53 single_joint_move

功能	驱动机械臂某一个轴旋转指定角度
注意	move_flag 等于 false 时调用有效
函数定义	def single_joint_move(self,axis,distance,speed)
参数	• int axis：关节号 1-4 正方向 • float distance：旋转角度(deg) • float speed：轴 3 为线速度(mm/s),其他为旋转速度(deg/s)
返回	0：正在执行其他指令，本次指令无效； 1：本次指令生效，机械臂开始运动； 2：设置速度小于等于零； 3：未初始化； 5：轴号参数错误； 6：伺服未开启； 11：手机端在控制； 101：传入参数 NOT A NUMBER； 102：发生碰撞，本次指令无效； 103：轴发生复位，需要重新初始化，本次指令无效；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.single_joint_move(4, 10,10)
```

```
if    ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
      print("机械臂指令无法执行")
```

3.1.2.54 xyz_move

功能	驱动机械臂沿某一方向运动指定距离
注意	move_flag 等于 false 时调用有效 运动轨迹为直线
函数定义	def xyz_move(self,direction,distance,speed)
参数	• int direction：1 x 轴正方向，2 y 轴正方向，3 z 轴正方向 • float distance：移动距离（mm） • float speed：移动速度（mm/s）
返回	0：正在执行其他指令，本次指令无效； 1：本次指令生效，机械臂开始运动； 2：设置速度小于等于零； 3：未初始化； 4：过程点无法到达； 5：方向参数错误； 6：伺服未开启； 7：该姿态无法完成直线运动； 11：手机端在控制； 101：传入参数 NOT A NUMBER； 102：发生碰撞，本次指令无效； 103：轴发生复位，需要重新初始化，本次指令无效；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.xyz_move(1, 2,10)
if   ret == 1:
      print("本次指令生效，机械臂开始运动")
else:
      print("机械臂指令无法执行")
```

3.1.2.55 set_Interpolation_position_buf_catch_count

功能	设置轨迹插补缓存区长度，减小或者增大停止或者暂停延时
注意	速度较快时，不能设置太小，会导致动作卡顿
函数定义	def set_Interpolation_position_buf_catch_count(self,count)
参数	• int count：插补缓存区长度，默认 200，区间 [32，4096]
返回	1：设置成功； 3：未初始化；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.set_Interpolation_position_buf_catch_count(32)
if   ret == 1:
      print("设置成功")
else:
      print("设置失败")
```

3.1.2.56 set_robot_buf_size

功能	设置手臂执行缓存区长度，减小或者增大停止或者暂停延时
注意	速度较快时，不能设置太小，会导致动作卡顿
函数定义	def set_robot_buf_size(self,bufsize)
参数	• int bufsize：插补缓存区长度，默认 850，区间 [32，900]

返回	1：设置成功； 3：未初始化；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.set_robot_buf_size(32)
if  ret == 1:
    print("设置成功")
else:
    print("设置失败")
```

3.1.2.57 set_robot_joint_torque_value

功能	设置各关节转矩设定值，限制电机输出转矩范围，来调节碰撞检测的灵敏度
注意	速度较快时，负载较重时，不能设置太小，会导致误报碰撞，具体大小根据需求调节设定
函数定义	def set_robot_joint_torque_value(self,jointnum,value)
参数	• int jointnum：轴编号 • int value：转矩设定值，[1000，20000]
返回	0：未连接； 1：设置成功； 2：轴号错误； 3：未初始化；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
ret = robot.set_robot_joint_torque_value(1,20000)
if  ret == 1:
    print("设置成功")
else:
    print("设置失败")
```

3.1.2.58 set_stop_io

功能	设置停止 I/O
注意	无
函数定义	def set_stop_io(self,io_number,io_value)
参数	• int io_number：I/O 号 • int io_value：I/0 信号值
返回	0：设置停止 I/O 成功；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.set_stop_io(1,0)
```

3.1.2.59 com485_init

功能	485 夹爪手动初始化
注意	无
函数定义	def com485_init(self)
参数	无
返回	1：夹爪手动初始化成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_init()
```

3.1.2.60 com485_set_rotation_speed

功能	485 夹爪设置旋转速度
注意	无
函数定义	def com485_set_rotation_speed(self,speed)
参数	• float speed：旋转速度
返回	1：夹爪设置旋转速度成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_set_rotation_speed(360)
```

3.1.2.61 com485_set_clamping_speed

功能	485 夹爪设置夹持速度
注意	无
函数定义	def com485_set_clamping_speed(self,speed)
参数	• float speed：夹持速度

返回	1：夹爪设置夹持速度成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_set_clamping_speed(50)
```

3.1.2.62 com485_set_clamping_distance

功能	485 夹爪设置夹持距离
注意	无
函数定义	def com485_set_clamping_distance(self,distance)
参数	• float distance：夹持距离
返回	1：夹爪设置夹持距离成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_set_clamping_distance(10)
```

3.1.2.63 com485_get_clamping_distance

功能	485 夹爪获取夹持距离
注意	无
函数定义	def com485_get_clamping_distance(self)
参数	• 无
返回	夹持距离

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_get_clamping_distance()
```

3.1.2.64 com485_set_rotation_angle

功能	485 夹爪设置绝对旋转角度
注意	无
函数定义	def com485_set_rotation_angle(self,angle)
参数	• float angle：绝对旋转角度
返回	1：夹爪设置绝对旋转角度成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_set_rotation_angle(90)
```

3.1.2.65 com485_set_relative_rotation_angle

功能	485 夹爪设置相对旋转角度
注意	无
函数定义	def com485_set_relative_rotation_angle(self,angle)
参数	• float angle：相对旋转角度
返回	1：夹爪设置相对旋转角度成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_set_relative_rotation_angle(90)
```

3.1.2.66 com485_get_rotation_angle

功能	485 夹爪获取旋转角度
注意	无
函数定义	def com485_get_rotation_angle(self)
参数	无

返回	旋转角度

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_get_rotation_angle()
```

3.1.2.67 com485_get_clamping_state

功能	485 夹爪获取夹持状态
注意	无
函数定义	def com485_get_clamping_state(self)
参数	无
返回	0：到位； 1：运动中； 2：夹持； 3：掉落；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_get_clamping_state()
```

3.1.2.68 com485_get_rotation_state

功能	485 夹爪获取旋转状态
注意	无
函数定义	def com485_get_rotation_state(self)
参数	无
返回	0：到位； 1：旋转中；

	2：旋转受阻； 3：掉落； 4：堵转停转；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_get_rotation_state()
```

3.1.2.69 com485_set_clamping_current

功能	485 夹爪设置夹持电流
注意	无
函数定义	def com485_set_clamping_current(self,current)
参数	• float current：夹持电流
返回	1：夹爪设置夹持电流成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_set_clamping_current(0.2)
```

3.1.2.70 com485_set_rotation_current

功能	485 夹爪设置旋转电流
注意	无
函数定义	def com485_set_rotation_current(self,current)
参数	• float current：旋转电流
返回	1：夹爪设置旋转电流成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_set_rotation_current(0.8)
```

3.1.2.71 com485_get_clamping_current

功能	485 夹爪获取夹持电流
注意	无
函数定义	def com485_get_clamping_current(self)
参数	无
返回	夹持电流

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_get_clamping_current()
```

3.1.2.72 com485_get_rotation_current

功能	485 夹爪获取旋转电流
注意	无
函数定义	def com485_get_rotation_current(self)
参数	无
返回	旋转电流

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_get_rotation_current()
```

3.1.2.73 com485_suction_cup_control

功能	485 吸盘控制
注意	无
函数定义	def com485_suction_cup_control(self, type)
参数	• int type：控制类型 1：吸取 2：释放 3：按真空度设置吸取
返回	1：吸盘控制成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时；

	3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_suction_cup_control(1)
```

3.1.2.74 com485_min_vacuum_degree

功能	485 吸盘最小真空度设置
注意	无
函数定义	def com485_min_vacuum_degree(self,degree)
参数	• int degree：真空度
返回	1：吸盘最小真空度设置成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_min_vacuum_degree(10)
```

3.1.2.75 com485_max_vacuum_degree

功能	485 吸盘最大真空度设置
注意	无
函数定义	def com485_max_vacuum_degree(self,degree)
参数	• int degree：真空度
返回	1：吸盘最大真空度设置成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_max_vacuum_degree(50)
```

3.1.2.76 com485_get_draw_state

功能	485 吸盘获取吸取状态
注意	无
函数定义	def com485_get_draw_state(self)
参数	无

返回	0：初始化或者释放物体； 1：吸住物体； 2：未吸住物体或者物体掉落；

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.com485_get_draw_state()
```

3.1.2.77 get_error_code

功能	获取错误信息
注意	无
函数定义	def get_error_code(self)
参数	无
返回	错误信息，详细见错误表

代码示例：

```
robot = HitbotInterface(24) #传入机械臂 id 号，接口实例化
return robot.get_error_code()
```

慧灵科技(深圳)有限公司
Huiling-tech Robotic Co.,Ltd

电话 0755-36382405
邮箱 hitbot@hitbot.cc
网址 www.hitbot.cc
地址 广东省深圳市宝安区西乡街道南昌社区航城大道
华丰国际机器人产业园B栋六层601-605